

Series Tech Limited Software Engineering Handbook

Complete Operational Standards, Git Governance, Language Rules, Security & Decision Matrix

1. Purpose & Scope

This document describes the complete software engineering practices at **Series Tech Limited**, covering development workflow, coding standards, Git workflow, branching strategy, pull requests, code review, version control, documentation standards, Agile methodology, sprint planning, security protocols, hotfix engineering, and decision matrices. It is written so that any engineer at Series Tech Limited can understand the engineering practices fully without needing another document.

Scope: This document applies to all employees in the Software Development department of Series Tech Limited, as well as engineers in Artificial Intelligence, Cloud Engineering, Data Engineering, and Quality Assurance who contribute code to Series Tech Limited or client repositories across the Hyderabad head office, Pune Enterprise Solutions hub, and hybrid remote workforces.

2. Introduction

Series Tech Limited follows a consistent set of software engineering practices across all projects to ensure code quality, maintainability, and predictable delivery. Whether an engineer is working in the Hyderabad head office, the Pune Enterprise Solutions hub, or remotely under the hybrid working model, the same development workflow and standards apply.

3. Development Workflow

The standard development workflow at Series Tech Limited begins with a requirement being groomed and added to the sprint backlog, followed by an engineer picking up the task, creating a feature branch, writing code and unit tests, opening a pull request, undergoing code review, and finally merging the change once approved. This development workflow applies consistently across all Series Tech Limited projects.

4. Coding Standards by Language

All code merged into Series Tech Limited repositories must follow language-specific conventions and pass configured static analysis/formatting tools prior to submission.

4.1 Java / Spring Boot

- **Naming Conventions:** Follow standard Java CamelCase conventions (PascalCase for classes, camelCase for methods/variables, UPPER_SNAKE_CASE for constants).
- **Formatting & Tooling:** Code must pass Checkstyle and SpotBugs rules. Formatted using Google Java Style Guide rules.
- **Architecture Standards:** Strict separation of Controller, Service, and Repository layers. Business logic must reside strictly in service layers.

4.2 Python / Django

- **Naming Conventions:** Follow PEP 8 guidelines (snake_case for functions/variables, PascalCase for classes/models, UPPER_SNAKE_CASE for constants).
- **Formatting & Tooling:** Enforced using Black, Flake8, and isort. Type hints are mandatory for public interface functions.
- **Django Rules:** Keep views light using Django REST Framework (DRF) serializers, handle database transactions explicitly, and avoid fat models.

4.3 JavaScript / React

- **Naming Conventions:** PascalCase for React Components and JSX files, camelCase for utilities, state variables, and custom hooks (e.g., useAuth).
- **Formatting & Tooling:** Strict ESLint and Prettier integration. Zero warnings permitted during build phase.
- **Component Guidelines:** Prefer functional components with hooks over class components. Maintain clean prop-types or TypeScript definitions.

4.4 SQL & Database Operations

- **Naming Conventions:** UPPERCASE for SQL keywords (SELECT, JOIN, WHERE), snake_case for table names and column names (e.g., user_accounts, created_at).
- **Formatting & Rules:** Explicit column selection is required (never use SELECT * in production code). Schema changes must be managed via version-controlled migration scripts (Liquibase or Django migrations).

5. Branch Naming & Commit Conventions

Series Tech Limited follows a trunk-based branching strategy with short-lived feature branches linked to JIRA/issue tracker IDs.

5.1 Branch Naming Format

feature/STL-123-user-login
bugfix/STL-456-payment-error
hotfix/STL-789-production-outage

5.2 Conventional Commits

Commit messages must follow Conventional Commit specifications to enable automated changelog generation:

- feat(auth): add JWT authentication endpoint
- fix(payment): resolve null pointer in checkout handler
- docs(readme): update API setup instructions
- refactor(db): optimize query performance on user search

6. Pull Request Lifecycle & Review Assignment

Every change enters production through a structured 7-stage lifecycle:

Draft PR → Automated Checks → Reviewer Assignment → Code Review → Approval → Merge → Deployment

6.1 Reviewer Assignment Matrix

Change Classification	Required Reviewers	Approval Threshold
Standard Feature / Bugfix	Peer Software Engineer	1 Engineer Approval
High-Risk / Core Module (Auth, Payments, Security)	Senior Engineer + Tech Lead / Architect	2 Engineer Approvals
Database Schema Migration	Database Engineer / Lead Developer	1 DB/Lead Approval + Passing Migration Check
Infrastructure / CI/CD Pipeline	DevOps / Cloud Infrastructure Specialist	1 DevOps Specialist Approval

7. Security & Dependency Management

Security is embedded into every layer of the engineering lifecycle at Series Tech Limited:

- **Software Composition Analysis (SCA):** Automated daily scanning of third-party libraries using Snyk / OWASP Dependency-Check. Critical CVEs block build pipelines immediately.
- **Secret Scanning:** Automated pre-commit hooks and platform scanners (e.g., GitGuardian) block commits containing API keys, private certificates, or database credentials.
- **Dependency Update Policy:** Minor and patch dependency updates are scheduled every sprint. High/Critical vulnerability updates must be patched within 48 hours of publication.
- **Discovered Vulnerability Protocol:** Security vulnerabilities found in internal code must be logged directly with the Cybersecurity Lead, remediated in a dedicated branch, and verified prior to production merge.

8. Incident & Hotfix Development Workflow

When an urgent defect affects production operations, engineers follow the expedited Hotfix workflow:

- **When to Use a Hotfix Branch:** Reserved exclusively for Severity 1 (Sev-1) or Severity 2 (Sev-2) production incidents requiring immediate fix outside standard 2-week sprint releases.
- **Emergency Fix Execution:** Hotfix branches (hotfix/STL-XXX-desc) are cut directly from the release tag or main branch.
- **Testing Requirements:** Must pass automated unit tests and a targeted regression test suite in the staging environment before deployment.
- **Post-Release Review:** A post-mortem (RCA) review is conducted within 48 hours, and hotfix changes are immediately back-merged into active feature branches to prevent regression.

9. Version Control & Documentation Standards

All source code, configuration files, and infrastructure-as-code scripts must be stored in Git. Every project must maintain a comprehensive README.md detailing setup instructions, environment requirements, and architecture summaries. New API endpoints or architectural shifts must be documented in the central engineering documentation repository.

10. Agile Methodology & Sprint Planning

Projects follow Scrum with standard two-week sprint cycles. Sprint planning sessions establish team capacity, evaluate backlog priorities, estimate story points using Fibonacci sequences, and define clear sprint goals. Sprint retrospectives occur at the end of each cycle to drive continuous process improvement.

11. Approval Workflow & Policy Summary

Category	Requirement / Standard
Standard Pull Request	At least 1 engineer approval required
High-Risk / Critical PR	2 engineer approvals required (e.g., auth, payments)
Main Branch Protection	Direct commits strictly prohibited; PRs & automated checks mandatory
Test Coverage Policy	PR cannot merge unless project minimum coverage threshold (80%) is met
Production Release	Sign-off from Project Lead, QA Lead, and Client Stakeholder (if client-facing)

12. Employee Scenarios & Common Mistakes

- **Scenario 1 (Divergence):** An engineer's feature branch remains open for two weeks. *Resolution:* Work must be broken into smaller pull requests and merged within a few days.
- **Scenario 2 (Superficial Review):** A reviewer leaves only stylistic comments. *Resolution:* Reviewers must also check for correctness, test coverage, and security implications.
- **Scenario 3 (Leaked Secret):** An API key is accidentally committed. *Resolution:* Report immediately, rotate the key at the source, and clean Git commit history per Cybersecurity policy.

13. Frequently Asked Questions (30 Standard FAQs)

1. What branching strategy does Series Tech Limited use?

A trunk-based branching strategy with short-lived feature branches.

2. How many approvals does a standard pull request need?

At least one engineer approval.

3. How many approvals does a high-risk pull request need?

Two engineer approvals.

4. Can I commit directly to the main branch?

No, all changes must go through a pull request.

5. What Agile methodology does Series Tech Limited follow?

Scrum, with sprints typically two weeks long.

6. Who attends sprint planning?

The Scrum Master, Product Owner, and the cross-functional development team.

7. What must every pull request include?

A clear description, a reference to the corresponding ticket, and evidence of successful unit test execution.

8. Where should credentials be stored?

In the project's approved secrets management solution, never in version control.

9. What is required before a pull request can be merged?

Required approvals and passing automated checks, enforced by branch protection rules.

10. Who signs off on a major release?

The project lead, the Quality Assurance lead, and the client stakeholder for client-facing changes.

11. What should a code reviewer check for?

Correctness, readability, coding standards, test coverage, and security issues.

12. How long should a feature branch stay open?

A few days, to avoid significant divergence from the main branch.

13. What documentation is required for every project?

A README file describing purpose, setup instructions, and architecture overview.

14. Can interns contribute code to Series Tech Limited repositories?

Yes, interns follow the same pull request and code review process as full-time engineers.

15. Can contractors access Series Tech Limited Git repositories?

Yes, based on project team membership and access controls.

16. What happens if unit test coverage is below the required threshold?

The pull request cannot be merged until coverage meets the project's minimum threshold.

17. How are commit messages expected to be written?

Clear and descriptive, explaining the purpose of the change.

18. What is trunk-based development?

A branching strategy where the main branch always reflects a stable, deployable state, with short-lived feature branches.

19. Who is the Scrum Master responsible for?

Facilitating the Scrum process, including sprint planning, daily stand-ups, and removing impediments.

20. What happens during sprint planning?

The team reviews the backlog, estimates effort, and commits to a sprint goal.

21. What is the role of the Product Owner?

Prioritizing the backlog and representing business and client requirements to the development team.

22. Can a pull request be merged without passing automated checks?

No, branch protection rules block merging until checks pass.

23. What happens if I accidentally commit a secret?

Report it immediately so the credential can be rotated and the commit history cleaned.

24. Is linting mandatory before submitting a pull request?

Yes, code must be linted using the project's configured tool.

25. What is the standard sprint length at Series Tech Limited?

Typically two weeks.

26. Who approves deployment to production?

As described in the Series Tech Limited Quality Assurance and DevOps document, following the release management process.

27. Can reviewers reject a pull request?

Yes, if it does not meet coding standards, lacks adequate tests, or has correctness or security concerns.

28. What is the internal Git platform used by Series Tech Limited?

A centrally hosted Git platform with project-based access control.

29. What is expected in a pull request description?

A clear explanation of the change and a reference to the corresponding ticket.

30. Who resolves disagreements during code review?

The engineering manager or team lead, if the engineers cannot reach agreement themselves.

14. Engineering Standards & Development Decision Matrix

This section serves as a direct, highly retrievable decision matrix for common operational questions across systems and workflows:

Question: Can I commit directly to main?

Answer: No. All changes require a pull request. Direct commits are blocked by automated branch protection rules.

Question: How many approvals does a payment-related PR require?

Answer: Two engineer approvals because payment functionality is classified as high-risk.

Question: Where should secrets be stored?

Answer: Only in the approved secrets management solution (HashiCorp Vault or AWS Secrets Manager), never in version control repositories.

Question: When should I create a hotfix branch?

Answer: When an urgent production issue requires an expedited fix outside the normal feature development workflow.

Question: What should I do if a third-party dependency has a High or Critical CVE vulnerability?

Answer: Patch or upgrade the dependency within 48 hours according to the Dependency Update Policy.

Question: Who must review database schema migration PRs?

Answer: A Database Engineer or Lead Developer, in addition to passing automated migration checks.

15. Revision History

Version	Date	Summary of Changes	Approved By
1.0	January 2025	First publication of the Series Tech Limited Software Engineering Handbook	Head of Software Development
1.1	January 2026	Updated code review approval thresholds for high-risk changes	Head of Software Development
2.0	July 2026	Added language-specific coding standards, PR reviewer assignment matrix, security scanning, hotfix workflows, and decision matrix.	Engineering Architecture Board